

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

**Oleh: Anggraeni Dwi Zahra
NIM. 2410817220018**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : AnggraeniDwi Zahra
NIM : 2410817220018

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code	7
B. Output Program	24
C. Pembahasan.....	27
D. Tautan Git	33

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Utama - Id).....	24
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose (Halaman Detail - Id)	25
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Utama - En)	25
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML (Halaman Detail - En)	26

DAFTAR TABEL

Tabel 1. Source Code MainActivity Compose Soal 1	7
Tabel 2. Source Code ApiService Compose Soal 1	8
Tabel 3. Source Code MovieResponse Compose Soal 1	9
Tabel 4. Source Code MovieEntity Compose Soal 1	10
Tabel 5. Source Code MovieDao Compose Soal 1	10
Tabel 6. Source Code AppDatabase Compose Soal 1	10
Tabel 7. Source Code MovieRepository Compose Soal 1	11
Tabel 8. Source Code LanguageScreen Compose Soal 1	13
Tabel 9. Source Code MovieScreens Compose Soal 1	15

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

COMPOSE

MainActivity.kt

Tabel 1. Source Code MainActivity Compose Soal 1

```
1 package com.example.modul3compose
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.viewModels
7 import com.example.modul3compose.data.AppDatabase
8 import com.example.modul3compose.data.MovieRepository
9 import com.example.modul3compose.data.NetworkClient
10 import
11 com.example.modul3compose.ui.theme.Modul3ComposeTheme
12 import timber.log.Timber
13
14 class MainActivity : ComponentActivity() {
15     private val apiKey =
16     "4b2fdca754cffe9b9c5c04c46b62bd10"
17     private val viewModel: MovieViewModel by viewModels
18     {
19         val database =
20         AppDatabase.getDatabase(applicationContext)
21         val repository =
22         MovieRepository(NetworkClient.apiService,
23         database.movieDao(), apiKey)
24         MovieViewModelFactory(repository)
25     }
26
27     override fun onCreate(savedInstanceState: Bundle?) {
28         super.onCreate(savedInstanceState)
29
30         val prefs = getSharedPreferences(
31             "app_prefs",
32             MODE_PRIVATE
33         )
34
35         val savedLanguage = prefs.getString(
36             "language",
37             "en"
38         ) ?: "en"
39
40         val locale = java.util.Locale(savedLanguage)
```

37	java.util.Locale.setDefault(locale)
38	
39	val config = resources.configuration
40	config.setLocale(locale)
41	
42	resources.updateConfiguration(
43	config,
44	resources.displayMetrics
45	)
46	
47	setContent {
48	Modul3ComposeTheme {
49	MovieApp(viewModel = viewModel)
50	}
51	}
52	
53	Timber.d("DEBUG_LOG: MainActivity initialized
54	successfully with TMDB API Integration")
55	}

ApiService.kt

Tabel 2. Source Code ApiService Compose Soal 1

1	package com.example.modul3compose.data
2	
3	import retrofit2.http.GET
4	import retrofit2.http.Path
5	import retrofit2.http.Query
6	
7	interface ApiService {
8	@GET("movie/popular")
9	suspend fun getPopularMovies(
10	@Query("api_key") apiKey: String,
11	@Query("language") language: String = "en-US",
12	@Query("page") page: Int = 1
13	): MovieResponse
14	@GET("movie/{movie_id}/videos")
15	suspend fun getMovieVideos(
16	@Path("movie_id") movieId: Int,
17	@Query("api_key") apiKey: String,
18	@Query("language") language: String = "en-US"
19	): VideoResponse
20	}

MovieResponse.kt

Tabel 3. Source Code MovieResponse Compose Soal 1

```
1 package com.example.modul3compose.data
2
3 import kotlinx.serialization.SerialName
4 import kotlinx.serialization.Serializable
5
6 @Serializable
7 data class MovieResponse (
8     @SerialName("results")
9     val results: List<MovieDto>
10 )
11
12 @Serializable
13 data class MovieDto(
14     @SerialName("id")
15     val id: Int,
16     @SerialName("title")
17     val title: String,
18     @SerialName("overview")
19     val overview: String,
20     @SerialName("poster_path")
21     val posterPath: String?,
22     @SerialName("release_date")
23     val releaseDate: String,
24     @SerialName("vote_average")
25     val voteAverage: Double
26 )
27
28 @Serializable
29 data class VideoResponse(
30     @SerialName("results")
31     val results: List<VideoDto>
32 )
33
34 @Serializable
35 data class VideoDto(
36     @SerialName("key")
37     val key: String,
38     @SerialName("name")
39     val name: String,
40     @SerialName("site")
41     val site: String,
42     @SerialName("type")
43     val type: String
44 )
45
```

MovieEntity.kt

Tabel 4. Source Code *MovieEntity* Compose Soal 1

```
1 package com.example.modul3compose.data
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "movies")
7 data class MovieEntity(
8     @PrimaryKey
9     val id: Int,
10    val title: String,
11    val overview: String,
12    val posterPath: String?,
13    val releaseDate: String,
14    val voteAverage: Double
15 )
```

MovieDao.kt

Tabel 5. Source Code *MovieDao* Compose Soal 1

```
1 package com.example.modul3compose.data
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface MovieDao {
11     @Query("SELECT * FROM movies")
12     fun getAllMovies(): Flow<List<MovieEntity>>
13
14     @Insert(onConflict = OnConflictStrategy.REPLACE)
15     suspend fun insertMovies(movies: List<MovieEntity>)
16
17     @Query("DELETE FROM movies")
18     suspend fun clearAllMovies()
19 }
```

AppDatabase.kt

Tabel 6. Source Code *AppDatabase* Compose Soal 1

```
1 package com.example.modul3compose.data
2
3 import android.content.Context
```

```

4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7
8 @Database(entities = [MovieEntity::class], version = 1,
9 exportSchema = false)
10 abstract class AppDatabase : RoomDatabase() {
11     abstract fun movieDao(): MovieDao
12
13     companion object {
14         @Volatile
15         private var INSTANCE: AppDatabase? = null
16
17         fun getDatabase(context: Context): AppDatabase {
18             return INSTANCE ?: synchronized(this) {
19                 val instance = Room.databaseBuilder(
20                     context.applicationContext,
21                     AppDatabase::class.java,
22                     "movie_database"
23                 ).build()
24                 INSTANCE = instance
25                 instance
26             }
27         }
28     }
29 }

```

MovieRepository.kt

Tabel 7. Source Code MovieRepository Compose Soal 1

```

1 package com.example.modul3compose.data
2
3 import kotlinx.coroutines.Dispatchers
4 import kotlinx.coroutines.flow.Flow
5 import kotlinx.coroutines.flow.flow
6 import kotlinx.coroutines.flow.flowOn
7
8 class MovieRepository(
9     private val apiService: ApiService,
10     private val movieDao: MovieDao,
11     private val apiKey: String
12 ) {
13     fun getPopularMovies():
14     Flow<Resource<List<MovieEntity>>> = flow {
15         emit(Resource.Loading)
16
17         try {

```

```

17         val response =
apiService.getPopularMovies(apiKey = apiKey)
18         val remoteMovies = response.results
19         val movieEntities = remoteMovies.map { dto:
MovieDto ->
20             MovieEntity(
21                 id = dto.id,
22                 title = dto.title,
23                 overview = dto.overview,
24                 posterPath = dto.posterPath,
25                 releaseDate = dto.releaseDate,
26                 voteAverage = dto.voteAverage
27             )
28         }
29
30         movieDao.clearAllMovies()
31         movieDao.insertMovies(movieEntities)
32
33     } catch (e: Exception) {
34         emit(Resource.Error(message = "Gagal memuat
data terbaru. Menampilkan data lokal.", throwable = e))
35     }
36
37     movieDao.getAllMovies().collect { localMovies ->
38         emit(Resource.Success(localMovies))
39     }
40 }.flowOn(Dispatchers.IO)
41
42 suspend fun getTrailerUrl(movieId: Int): String? {
43     val response = apiService.getMovieVideos(
44         movieId = movieId,
45         apiKey = apiKey
46     )
47
48     val trailer = response.results.firstOrNull {
49         it.site == "YouTube" &&
50         it.type == "Trailer"
51     }
52
53     return trailer?.let {
54         "https://www.youtube.com/watch?v=${it.key}"
55     }
56 }
57 }

```

LanguageScreen.kt

Tabel 8. Source Code LanguageScreen Compose Soal 1

```
1 package com.example.modul3compose
2
3 import android.content.res.Configuration
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Column
6 import androidx.compose.foundation.layout.Spacer
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.fillMaxWidth
9 import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.rememberScrollState
12 import androidx.compose.foundation.verticalScroll
13 import androidx.compose.material3.Button
14 import androidx.compose.material3.MaterialTheme
15 import androidx.compose.material3.Text
16 import androidx.compose.material3.TextButton
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.platform.LocalContext
21 import androidx.compose.ui.res.stringResource
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import androidx.lifecycle.viewmodel.compose.viewModel
25 import java.util.Locale
26
27 @Composable
28 fun LanguageScreen(
29     onBackClick: () -> Unit,
30     languageViewModel: LanguageViewModel = viewModel()
31 ) {
32     val context = LocalContext.current
33
34     Column(
35         modifier = Modifier
36             .fillMaxSize()
37             .padding(16.dp)
38             .verticalScroll(rememberScrollState()),
39         verticalArrangement = Arrangement.Center,
40         horizontalAlignment =
41 Alignment.CenterHorizontally
42     ) {
43         Text(
44             text = stringResource(id =
45 R.string.language_selector),
46             style = MaterialTheme.typography.titleLarge,
```

```

45         fontWeight = FontWeight.Bold
46     )
47
48     Spacer(modifier = Modifier.height(32.dp))
49
50     languageViewModel.LanguageList.forEach {
language ->
51         Button(
52             onClick = {
53
54                 val prefs =
context.getSharedPreferences(
55                     "app_prefs",
56                     android.content.Context.MODE_PRIVATE
57                 )
58
59                 prefs.edit()
60                     .putString("language",
language.code)
61                     .apply()
62
63                 val locale = Locale(language.code)
64                 Locale.setDefault(locale)
65
66                 val config = Configuration()
67                 config.setLocale(locale)
68
69                 context.resources.updateConfiguration(
70                     config,
71                     context.resources.displayMetrics
72                 )
73
74                 onBackPressed()
75             },
76             modifier = Modifier
77                 .fillMaxWidth()
78                 .padding(horizontal = 32.dp)
79         ) {
80             Text(text = language.name)
81         }
82
83         Spacer(modifier = Modifier.height(12.dp))
84     }
85     Spacer(modifier = Modifier.height(20.dp))
86
87     TextButton(onClick = onBackPressed) {
88         Text(text = stringResource(id =
R.string.back_button))

```

89	}
90	}
91	}

MovieScreens.kt

Tabel 9. Source Code MovieScreens Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.clickable
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Box
8	import androidx.compose.foundation.layout.Column
9	import androidx.compose.foundation.layout.PaddingValues
10	import androidx.compose.foundation.layout.Row
11	import androidx.compose.foundation.layout.Spacer
12	import androidx.compose.foundation.layout.fillMaxSize
13	import androidx.compose.foundation.layout.fillMaxWidth
14	import androidx.compose.foundation.layout.height
15	import androidx.compose.foundation.layout.padding
16	import androidx.compose.foundation.layout.size
17	import androidx.compose.foundation.layout.width
18	import androidx.compose.foundation.lazy.LazyColumn
19	import androidx.compose.foundation.lazy.LazyRow
20	import androidx.compose.foundation.lazy.items
21	import androidx.compose.foundation.rememberScrollState
22	import
	androidx.compose.foundation.shape.RoundedCornerShape
23	import androidx.compose.foundation.verticalScroll
24	import androidx.compose.material.icons.Icons
25	import androidx.compose.material.icons.filled.Language
26	import androidx.compose.material3.Button
27	import androidx.compose.material3.Card
28	import androidx.compose.material3.CardDefaults
29	import
	androidx.compose.material3.CircularProgressIndicator
30	import androidx.compose.material3.Icon
31	import androidx.compose.material3.IconButton
32	import androidx.compose.material3.MaterialTheme
33	import androidx.compose.material3.Text
34	import androidx.compose.runtime.Composable
35	import androidx.compose.runtime.collectAsState
36	import androidx.compose.runtime.getValue
37	import androidx.compose.ui.Alignment
38	import androidx.compose.ui.Modifier
39	import androidx.compose.ui.draw.clip

```

40 import androidx.compose.ui.graphics.Color
41 import androidx.compose.ui.layout.ContentScale
42 import androidx.compose.ui.platform.LocalContext
43 import androidx.compose.ui.res.stringResource
44 import androidx.compose.ui.text.font.FontWeight
45 import androidx.compose.ui.text.style.TextAlign
46 import androidx.compose.ui.text.style.TextOverflow
47 import androidx.compose.ui.unit.dp
48 import androidx.compose.ui.unit.sp
49 import androidx.navigation.NavController
50 import androidx.navigation.compose.NavHost
51 import androidx.navigation.compose.composable
52 import androidx.navigation.compose.rememberNavController
53 import coil.compose.AsyncImage
54 import com.example.modul3compose.data.MovieEntity
55 import com.example.modul3compose.data.Resource
56 import timber.log.Timber
57 import androidx.compose.runtime.rememberCoroutineScope
58 import kotlinx.coroutines.launch
59
60 @Composable
61 fun MovieApp(viewModel: MovieViewModel) {
62     val navController = rememberNavController()
63     val moviesState by
64     viewModel.moviesState.collectAsState()
65
66     NavHost(navController = navController,
67 startDestination = "movie_list") {
68         composable("movie_list") {
69             when (val state = moviesState) {
70                 is Resource.Loading -> {
71                     Box(modifier =
72 Modifier.fillMaxSize(), contentAlignment =
73 Alignment.Center) {
74                         CircularProgressIndicator()
75                     }
76                 }
77                 is Resource.Success -> {
78                     MovieList(
79                         movies = state.data,
80                         navController = navController,
81                         viewModel = viewModel,
82                         onDetailClick = { movie ->
83                             viewModel.onMovieClicked(movie)
84                             navController.navigate("movie_detail/${movie.id}")
85                         }
86                     )
87                 }
88                 is Resource.Error -> {

```

```

85         Box(modifier =
Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
86             Text(text = "Error:
${state.message}", color = Color.Red, textAlign =
TextAlign.Center)
87         }
88     }
89 }
90 }
91
92     composable("movie_detail/{movieId}") {
backStackEntry ->
93         val movieId =
backStackEntry.arguments?.getString("movieId")?.toIntOrNull()
94         val movie = viewModel.getMovieById(movieId)
95         if (movie != null) {
96             MovieDetailScreen(
97                 movie = movie,
98                 onBackPressed = {
navController.popBackStack() }
99             )
100         }
101     }
102
103     composable("language_screen") {
104         LanguageScreen(onBackPressed = {
navController.popBackStack() })
105     }
106 }
107 }
108
109 @Composable
110 fun MovieList(
111     movies: List<MovieEntity>,
112     navController: NavController,
113     viewModel: MovieViewModel,
114     onDetailClick: (MovieEntity) -> Unit
115 ) {
116     LazyColumn(modifier = Modifier.fillMaxSize()) {
117         item {
118             HomeHeader(navController = navController)
119
120             Text(
121                 text =
stringResource(R.string.featured_movies),
122                 modifier = Modifier.padding(start =
16.dp, top = 8.dp, bottom = 8.dp),

```

```

123         style =
MaterialTheme.typography.titleLarge,
124         fontWeight = FontWeight.ExtraBold
125     )
126
127     LazyRow(
128         contentPadding =
PaddingValues(horizontal = 8.dp),
129         horizontalArrangement =
Arrangement.spacedBy(8.dp)
130     ) {
131         items(movies) { movie ->
132             MovieHighlightItem(
133                 movie = movie,
134                 onDetailClick = {
onDetailClick(movie) }
135             )
136         }
137     }
138
139     Spacer(modifier = Modifier.height(16.dp))
140
141     Text(
142         text =
stringResource(R.string.all_movies),
143         modifier = Modifier.padding(start =
16.dp, bottom = 8.dp),
144         style =
MaterialTheme.typography.titleMedium,
145         fontWeight = FontWeight.Bold
146     )
147 }
148
149 items(movies) { movie ->
150     MovieItemRow(
151         movie = movie,
152         viewModel = viewModel,
153         onDetailClick = { onDetailClick(movie) }
154     )
155 }
156 }
157 }
158
159 @Composable
160 fun HomeHeader(navController: NavController) {
161     Row(
162         modifier = Modifier
163             .fillMaxWidth()
164             .padding(16.dp),

```

```

165         horizontalArrangement = Arrangement.End,
166         verticalAlignment = Alignment.CenterVertically
167     ) {
168         IconButton(onClick = {
169             navController.navigate("language_screen") }) {
170             Icon(
171                 imageVector = Icons.Default.Language,
172                 contentDescription = "Change Language"
173             )
174         }
175     }
176
177 @Composable
178 fun MovieItemRow(
179     movie: MovieEntity,
180     viewModel: MovieViewModel,
181     onDetailClick: () -> Unit
182 ) {
183     val context = LocalContext.current
184     val scope = rememberCoroutineScope()
185
186     Card(
187         shape = RoundedCornerShape(16.dp),
188         modifier = Modifier
189             .fillMaxWidth()
190             .padding(8.dp),
191         elevation = CardDefaults.cardElevation(4.dp)
192     ) {
193         Row(
194             modifier = Modifier.padding(12.dp),
195             verticalAlignment =
196             Alignment.CenterVertically
197         ) {
198             AsyncImage(
199                 model =
200                 "https://image.tmdb.org/t/p/w500${movie.posterPath}",
201                 contentDescription = movie.title,
202                 modifier = Modifier
203                     .size(100.dp)
204                     .clip(RoundedCornerShape(12.dp)),
205                 contentScale = ContentScale.Crop
206             )
207
208             Spacer(modifier = Modifier.width(16.dp))
209
210             Column(modifier = Modifier.weight(1f)) {

```

```

211         Row(
212             modifier = Modifier.fillMaxWidth(),
213             horizontalArrangement =
Arrangement.SpaceBetween
214         ) {
215             Text(
216                 text = movie.title,
217                 fontWeight = FontWeight.Bold,
218                 modifier = Modifier.weight(1f),
219                 overflow = TextOverflow.Ellipsis
220             )
221
222             Text(
223                 text = "★
224             ${movie.voteAverage}",
225                 fontSize = 12.sp
226             )
227         }
228
229         Row(
230             modifier = Modifier.fillMaxWidth(),
231             horizontalArrangement =
Arrangement.SpaceBetween
232         ) {
233             Text(
234                 text = "Release:
235             ${movie.releaseDate}",
236                 fontSize = 11.sp,
237                 color = Color.Gray
238             )
239
240             Text(
241                 text =
stringResource(R.string.popular),
242                 fontWeight =
FontWeight.SemiBold,
243                 color = Color.Blue,
244                 fontSize = 13.sp
245             )
246         }
247
248         Spacer(modifier = Modifier.height(8.dp))
249
250         Row(
251             modifier = Modifier.fillMaxWidth(),
252             horizontalArrangement =
Arrangement.spacedBy(
253                 8.dp,
254                 Alignment.End

```

```

253         )
254     ) {
255
256         Button(
257             onClick = {
258                 scope.launch {
259                     try {
260                         val trailerUrl =
261                             viewModel.getTrailerUrl(movie.id)
262
263                         if (trailerUrl !=
264                             null) {
265                             val intent =
266                                 Intent(
267                                     Intent.ACTION_VIEW,
268                                     Uri.parse(trailerUrl)
269                                 )
270
271                             context.startActivity(intent)
272                         } catch (e: Exception) {
273                             Timber.e(
274                                 e,
275                                 "Gagal membuka
276                                 trailer"
277                             )
278                         }
279                     }
280                 ) {
281                     Text(stringResource(R.string.trailer))
282                 }
283
284                 Button(
285                     onClick = {
286                         onDetailClick()
287                     }
288                 ) {
289                     Text(stringResource(R.string.detail))
290                 }
291             }
292         }
293     }
294 }
295

```

```

296 @Composable
297 fun MovieHighlightItem(movie: MovieEntity,
298 onDetailClick: () -> Unit) {
299     Card(
300         shape = RoundedCornerShape(20.dp),
301         modifier = Modifier
302             .width(300.dp)
303             .padding(8.dp)
304             .clip(RoundedCornerShape(20.dp))
305             .clickable { onDetailClick() },
306         elevation = CardDefaults.cardElevation(6.dp)
307     ) {
308         Column {
309             AsyncImage(
310                 model =
311                 "https://image.tmdb.org/t/p/w500${movie.posterPath}",
312                 contentDescription = movie.title,
313                 modifier = Modifier
314                     .fillMaxWidth()
315                     .height(150.dp),
316                 contentScale = ContentScale.Crop
317             )
318             Text(
319                 text = movie.title,
320                 modifier = Modifier.padding(12.dp),
321                 fontWeight = FontWeight.Bold,
322                 style =
323                 MaterialTheme.typography.titleMedium
324             )
325         }
326     }
327 }
328
329 @Composable
330 fun MovieDetailScreen(movie: MovieEntity, onBackClick:
331 () -> Unit) {
332     Column(
333         modifier = Modifier
334             .fillMaxSize()
335             .verticalScroll(rememberScrollState())
336     ) {
337         Box {
338             AsyncImage(
339                 model =
340                 "https://image.tmdb.org/t/p/w500${movie.posterPath}",
341                 contentDescription = movie.title,
342                 modifier = Modifier
343                     .fillMaxWidth()
344                     .height(300.dp),

```

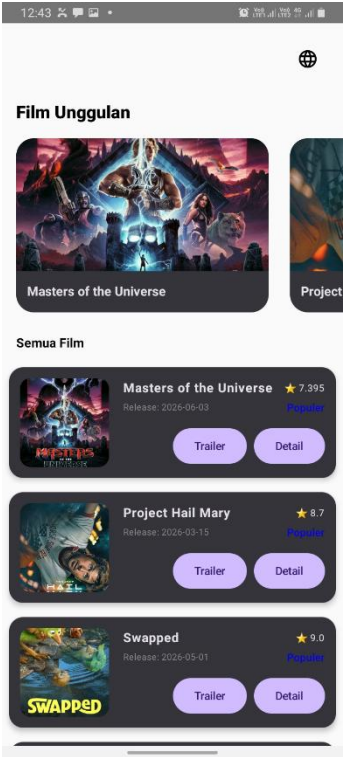
```

341         contentScale = ContentScale.Crop
342     )
343
344     Button(
345         onClick = onBackPressed,
346         modifier = Modifier.padding(16.dp),
347         shape = RoundedCornerShape(50)
348     ) {
349         Text(text = stringResource(id =
350 R.string.back_button))
351     }
352
353     Column(modifier = Modifier.padding(16.dp)) {
354         Row(
355             modifier = Modifier.fillMaxWidth(),
356             horizontalArrangement =
357 Arrangement.SpaceBetween,
358             verticalAlignment =
359 Alignment.CenterVertically
360         ) {
361             Text(text = movie.title, style =
362 MaterialTheme.typography.headlineMedium, fontWeight =
363 FontWeight.Bold, modifier = Modifier.weight(1f))
364             Spacer(modifier = Modifier.width(8.dp))
365             Text(text = "★ ${movie.voteAverage}",
366 style = MaterialTheme.typography.headlineSmall,
367 textAlign = TextAlign.End)
368         }
369
370         Text(text = "Release Date:
371 ${movie.releaseDate}", color = Color.Gray, fontSize =
372 16.sp)
373
374         Spacer(modifier = Modifier.height(24.dp))
375
376         Text(
377             text =
378 stringResource(R.string.overview),
379             style =
380 MaterialTheme.typography.titleMedium,
381             fontWeight = FontWeight.Bold
382         )
383         Text(
384             text = movie.overview,
385             style =
386 MaterialTheme.typography.bodyMedium,
387             color = Color.DarkGray
388         )

```

378	}
379	}
380	}

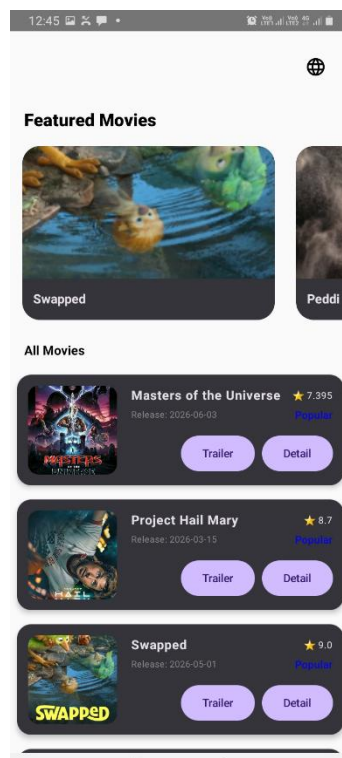
B. Output Program
A. Compose



Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Utama - Id)



Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose (Halaman Detail - Id)



Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Utama - En)



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML (Halaman Detail - En)

C. Pembahasan

COMPOSE

MainActivity.kt

File ini merupakan komponen utama yang menjalankan fungsi "onCreate(savedInstanceState: Bundle?)" untuk menangani siklus hidup aktivitas. Di dalamnya, terdapat inisialisasi "viewModel" menggunakan "viewModels" factory yang secara krusial menyiapkan "AppDatabase", "MovieDao", dan "MovieRepository" agar siap digunakan dengan TMDB API key. Selain itu, terdapat fungsi krusial untuk membaca konfigurasi bahasa dari "SharedPreferences" dengan nama "app_prefs" menggunakan parameter "language" dan nilai bawaan "en". SharedPreferences digunakan untuk menyimpan preferensi bahasa pengguna. Saat pengguna memilih bahasa Indonesia atau Inggris, kode bahasa disimpan menggunakan SharedPreferences. Pada saat aplikasi dijalankan kembali, aplikasi membaca preferensi tersebut dan menerapkan locale yang tersimpan sehingga bahasa yang dipilih tetap digunakan. Kode tersebut kemudian memperbarui konfigurasi locale sistem melalui "resources.updateConfiguration(config, resources.displayMetrics)". Terakhir, fungsi "setContent" memanggil tema "Modul3ComposeTheme" dan menginisialisasi navigasi dengan mengoper objek "viewModel" ke dalam composable "MovieApp".

ApiService.kt

File ini merupakan sebuah interface Retrofit yang mendefinisikan kontrak untuk melakukan pemanggilan jaringan (remote API request) ke server TMDB. Di dalamnya terdapat dua fungsi krusial, yaitu "getPopularMovies()" dengan anotasi "@GET(\"movie/popular\")" untuk mengambil daftar film populer, serta "getMovieVideos()" dengan anotasi "@GET(\"movie/{movie_id}/videos\")" untuk mengambil data video terkait film. Kedua fungsi ini menggunakan kata kunci "suspend" agar dapat dijalankan secara asinkron dalam coroutine tanpa memblokir UI thread. Parameter yang digunakan meliputi "@Query(\"api_key\")"

untuk autentikasi, "@Path("movie_id")" untuk menentukan ID film yang dicari, serta "@Query("language")" dan "@Query("page")" sebagai parameter opsional pelengkap request.

MovieResponse.kt

File ini berfungsi sebagai Data Transfer Object (DTO) yang merepresentasikan struktur data JSON dari remote API TMDB. File ini menggunakan anotasi "@Serializable" dari library KotlinX Serialization agar objek dapat dikonversi dari dan ke format JSON secara otomatis. Di dalamnya terdapat data class "MovieResponse" dan "VideoResponse" yang membungkus list data hasil request. Setiap properti di dalam data class seperti "MovieDto" dan "VideoDto" dilengkapi dengan anotasi "@SerializedName" yang berfungsi memetakan nama kunci (key) dari JSON API ke properti variabel Kotlin agar tetap konsisten meskipun format penulisan namanya berbeda.

MovieEntity.kt

File ini berisi "data class MovieEntity" yang mendefinisikan struktur skema tabel untuk database lokal Room. Kelas ini menggunakan anotasi "@Entity(tableName = "movies")" untuk mendeklarasikan bahwa objek ini akan disimpan sebagai baris data di dalam tabel bernama "movies". Bagian yang sangat krusial di sini adalah penggunaan anotasi "@PrimaryKey" pada parameter "id: Int", yang memastikan setiap data film memiliki pengidentifikasi unik dan mencegah terjadinya duplikasi data di dalam database lokal. Properti lain seperti "title", "overview", "posterPath", "releaseDate", dan "voteAverage" digunakan untuk menyimpan informasi film yang berhasil diunduh agar bisa diakses secara luring (offline).

MovieDao.kt

File ini merupakan sebuah interface Data Access Object (DAO) yang menggunakan anotasi "@Dao" untuk menjembatani kode aplikasi dengan operasi

database SQLite melalui Room. Di dalamnya terdapat tiga fungsi krusial: "getAllMovies()" yang menggunakan anotasi "@Query(\"SELECT * FROM movies\")" dengan tipe pengembalian "Flow<List<MovieEntity>>" untuk menghasilkan data stream yang reaktif. "insertMovies(movies: List<MovieEntity>)" yang menggunakan anotasi "@Insert(onConflict = OnConflictStrategy.REPLACE)" untuk menyisipkan data film baru sekaligus mengganti data lama jika terjadi konflik ID; serta "clearAllMovies()" dengan "@Query(\"DELETE FROM movies\")" untuk mengosongkan tabel. Fungsi manipulasi data di sini menggunakan kata kunci "suspend" agar aman dieksekusi di background thread.

AppDatabase.kt

File ini adalah kelas abstrak yang mewarisi "RoomDatabase" dan diberi anotasi "@Database" untuk mengonfigurasi entitas "MovieEntity" serta versi database yang digunakan. Kelas ini menyediakan fungsi abstrak "movieDao()" untuk mengekspos fungsi-fungsi query ke aplikasi. Bagian paling krusial dari file ini terletak pada companion object yang menerapkan pattern Singleton melalui variabel "@Volatile private var INSTANCE". Pola ini memastikan bahwa instansiasi database menggunakan "Room.databaseBuilder()" hanya terjadi satu kali di seluruh siklus hidup aplikasi melalui blok "synchronized(this)", sehingga menghemat penggunaan memori dan mencegah konflik akses file database.

MovieRepository.kt

File ini bertindak sebagai mediator data (*Single Source of Truth*) yang mengelola perpaduan data dari remote API ("ApiService") dan local storage ("MovieDao"). Fungsi utamanya adalah "getPopularMovies()" yang mengembalikan "Flow<Resource<List<MovieEntity>>>" dan dieksekusi pada "Dispatchers.IO" agar tidak mengganggu kinerja UI. Di dalam blok `flow`, aplikasi pertama-tama memancarkan "Resource.Loading", kemudian mencoba mengambil data terbaru dari API, memetakan "MovieDto" menjadi "MovieEntity", membersihkan

database lama via "clearAllMovies()", dan menyimpan data baru via "insertMovies()". Jika jaringan gagal, blok catch akan menangkap *exception* dan memancarkan *Resource.Error*. Namun, aplikasi akan tetap menjalankan perintah "movieDao.getAllMovies().collect" untuk memancarkan data lokal yang tersimpan di Room sebagai strategi *caching*. Selain itu, terdapat fungsi "getTrailerUrl(movieId: Int)" yang secara krusial mencari video bertipe "Trailer" dari platform "YouTube" untuk menghasilkan string URL video film.

LanguageScreen.kt

File ini berisi fungsi UI berbasis Compose bernama LanguageScreen yang berfungsi untuk menampilkan antarmuka pilihan bahasa aplikasi. Parameter yang diterima adalah "onBackClick: () -> Unit" untuk aksi navigasi kembali dan "languageViewModel" untuk memuat daftar bahasa. Di dalam komponen "Column" yang memiliki modifier "`verticalScroll`", terdapat perulangan "forEach" untuk merender tombol pilihan bahasa secara dinamis. Fungsi krusial di dalam blok klik ("onClick") tombol tersebut meliputi penyimpanan kode bahasa ke "SharedPreferences" secara asinkron menggunakan ".putString().apply()", pengaturan ulang objek *Locale*, serta pembaruan konfigurasi aplikasi global melalui "context.resources.updateConfiguration()" sebelum memicu fungsi navigasi `onBackClick()` untuk menerapkan perubahan tampilan secara instan.

MovieScreens.kt

File ini merupakan pusat komponen antarmuka (UI) berbasis Jetpack Compose yang merangkum seluruh navigasi dan layout aplikasi.

- a) **MovieApp**: Menginisialisasi "rememberNavController()" dan menyusun NavHost dengan rute awal "movie_list". Di dalamnya terdapat logika pengondisian "when(state)" untuk menampilkan "CircularProgressIndicator" saat memuat data, memanggil "MovieList" saat sukses, atau memunculkan pesan teks berwarna merah saat error.
- b) **MovieList**: Memanfaatkan komponen "LazyColumn" dan "LazyRow" yang

diisi fungsi "items()" untuk menyusun daftar film horizontal (*highlight*) dan vertikal (all movies) secara efisien tanpa menurunkan performa perangkat.

- c) **MovieItemRow**: Menampilkan item film individual menggunakan layout "Card", "Row", dan "Column". File ini secara krusial menggunakan library *Coil* melalui "AsyncImage" dengan parameter URL string gambar "https://image.tmdb.org/t/p/w500..." untuk memuat poster film dari internet. Di dalam komponen ini terdapat tombol trailer yang memanfaatkan *Implicit Intent* via coroutine "scope.launch" dengan memanggil "Intent(Intent.ACTION_VIEW, Uri.parse(trailerUrl))" agar dapat membuka aplikasi eksternal (seperti YouTube atau Browser) melalui perintah "context.startActivity(intent)".
- d) **MovieDetailScreen**: Layar penuh yang memanfaatkan parameter "MovieEntity" pilihan untuk menampilkan poster besar film, judul, rating bintang, tanggal rilis, beserta sinopsis lengkap (*overview*) dalam sebuah penampung yang dapat digulir.

Implementasi Chaching Strategy

Aplikasi ini menerapkan caching strategy berupa Network-First Cache menggunakan Room Database sebagai penyimpanan lokal. Ketika aplikasi dijalankan, data film terlebih dahulu diambil dari TMDB API. Jika proses pengambilan data berhasil, data lama pada Room akan diperbarui dengan data terbaru dari server. Selanjutnya aplikasi menampilkan data yang berasal dari Room Database. Apabila terjadi kegagalan jaringan atau API tidak dapat diakses, aplikasi tetap dapat menampilkan data yang sebelumnya telah tersimpan pada Room sehingga pengguna masih dapat melihat informasi film secara offline.

Strategi ini dipilih karena data film pada TMDB dapat berubah dari waktu ke waktu, sehingga aplikasi tetap berusaha memperoleh data terbaru dari server. Di sisi lain, Room digunakan sebagai cache lokal agar aplikasi tetap dapat digunakan ketika koneksi internet tidak tersedia serta mengurangi ketergantungan terhadap jaringan. Berikut adalah alur yang digunakan:

1. Bagian Penyimpanan Data API ke Room:

```
movieDao.insertMovies(movieEntities)
```

Potongan kode ini membuktikan bahwa data yang sukses diambil dari internet (TMDB API) langsung disimpan ke dalam Room Database sebagai cache lokal.

2. Bagian Pembacaan Data dari Room:

```
movieDao.getAllMovies()
```

Potongan kode ini membuktikan bahwa data yang mengalir dan ditampilkan pada layar aplikasi selalu bersumber dari Room Database.

3. Bagian Penanganan Saat Jaringan Gagal:

```
catch (e: Exception)
```

Blok penanganan error ini memastikan bahwa apabila koneksi ke remote API gagal atau terputus, data lama yang berada di dalam Room tetap dipertahankan sehingga aplikasi masih mampu memunculkan informasi film secara offline.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat:

<https://github.com/AnggraeniDwiZahra/Pemrograman-Mobile>